

Índice

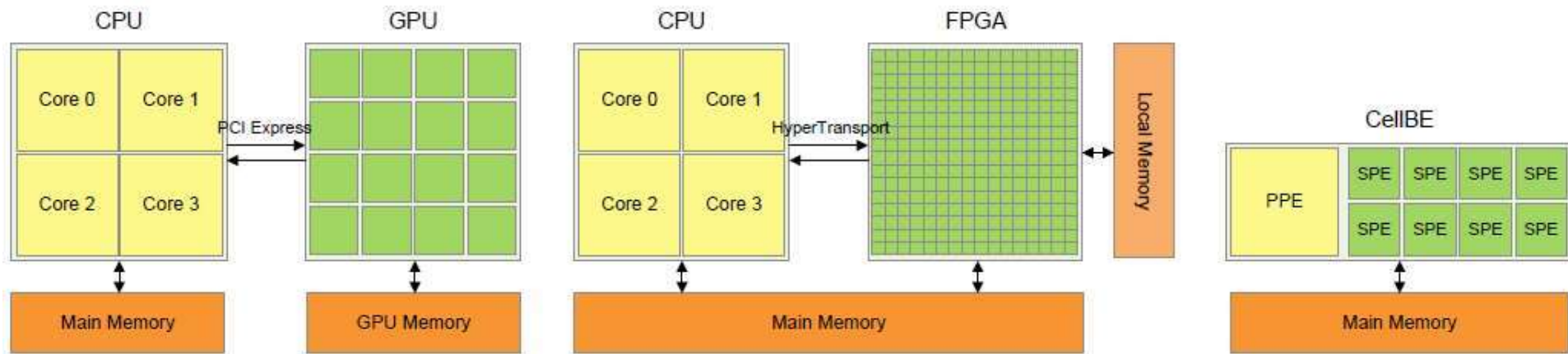
- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Índice

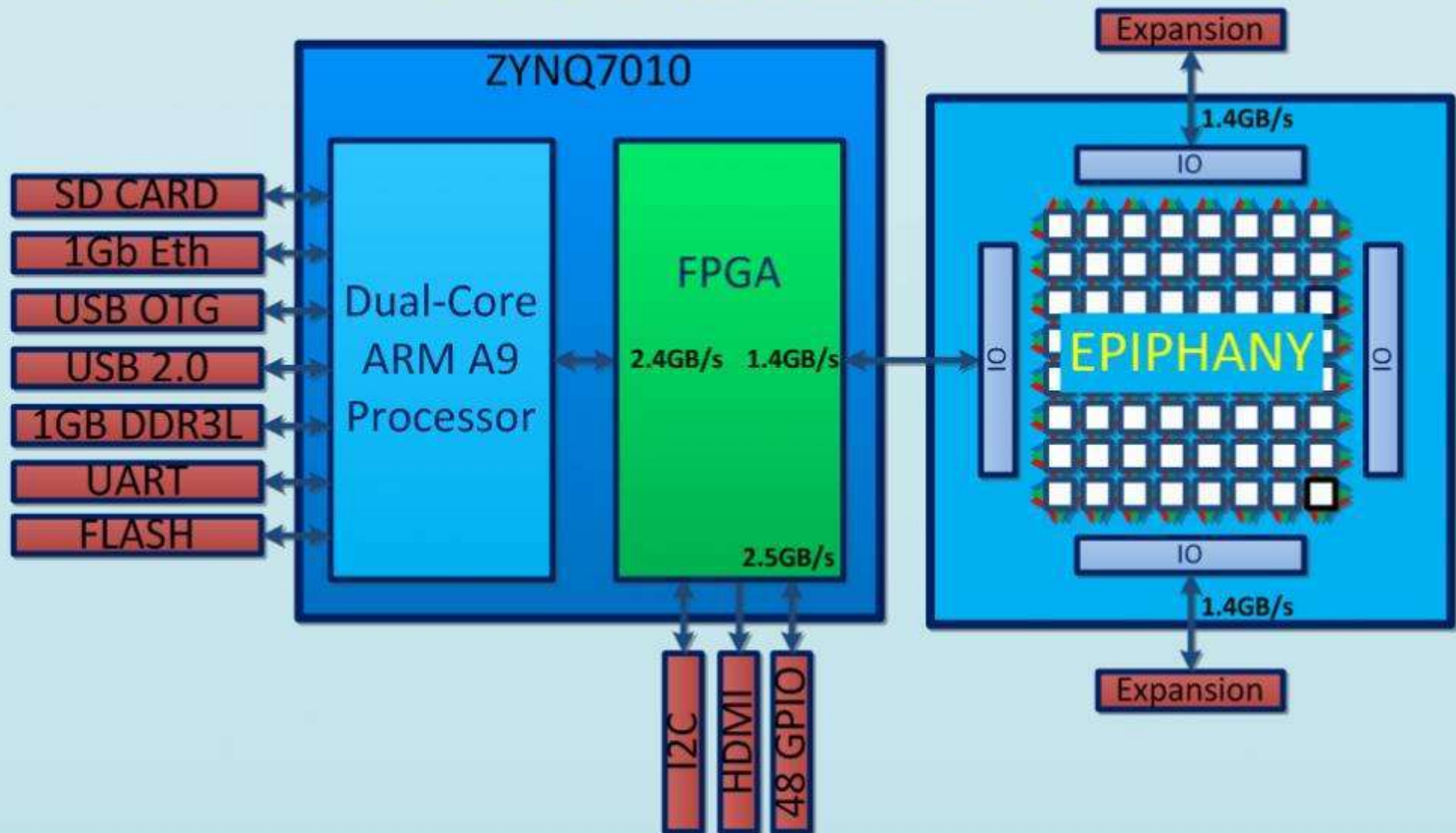
- **1. Computación Heterogénea**
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Computación Heterogénea

- Más de un procesador trabajando conjuntamente.
- Distintas arquitecturas de procesadores que se complementen.



PARALLELLA



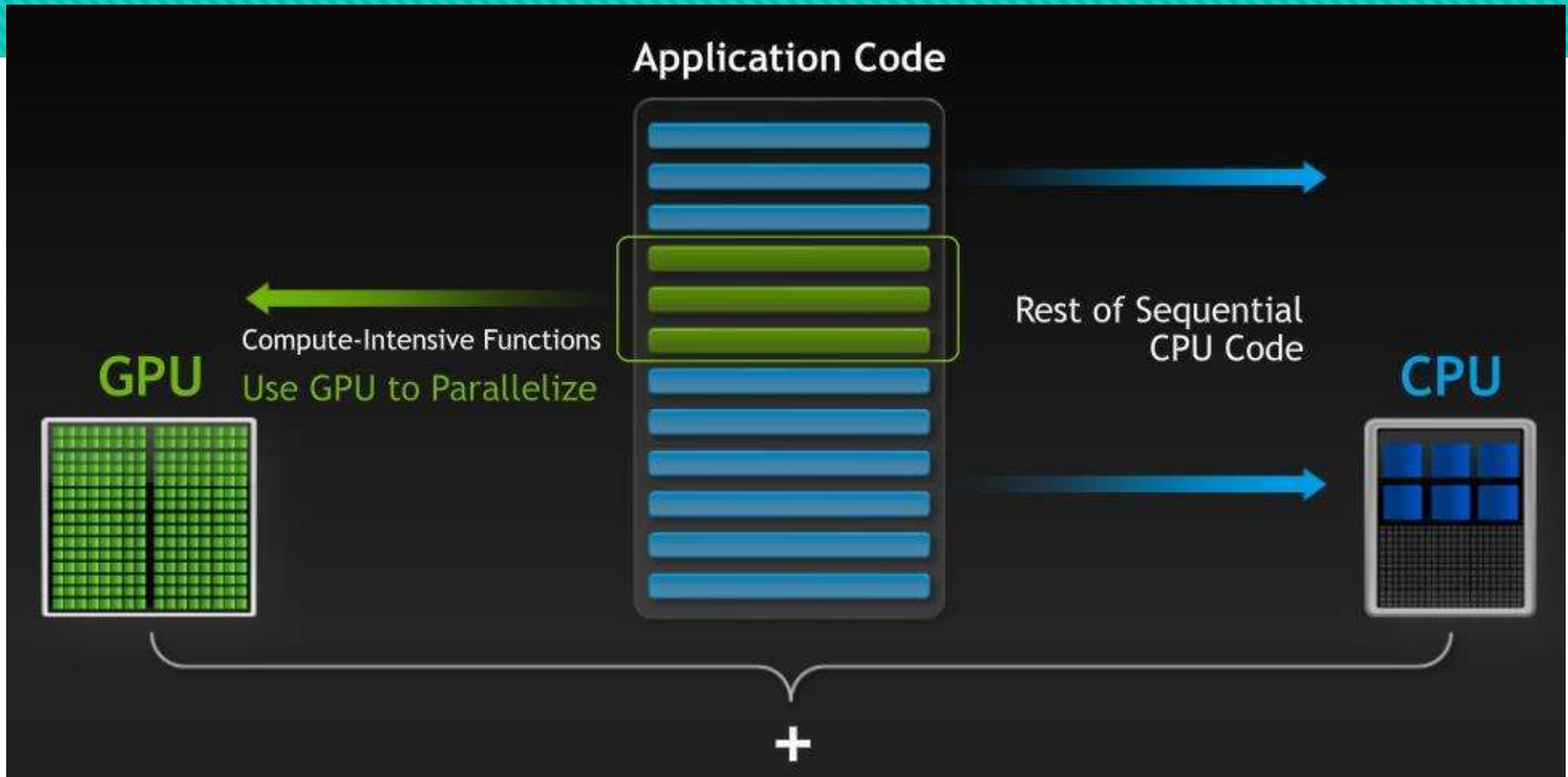
Computación Heterogénea

- Existen 4 niveles de paralelismo:
 - SISD – Single Instruction Single Data
 - SIMD – Single Instruction Multiple Data
 - MISD – Multiple Instruction Multiple Data
 - MIMD – Multiple Instruction Multiple Data
- SPMD – Single Program Multiple Data

Índice

- 1. Computación Heterogénea
- **2. Arquitectura Nvidia**
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Arquitectura NVIDIA



Arquitectura NVIDIA

Código serie



Código paralelo



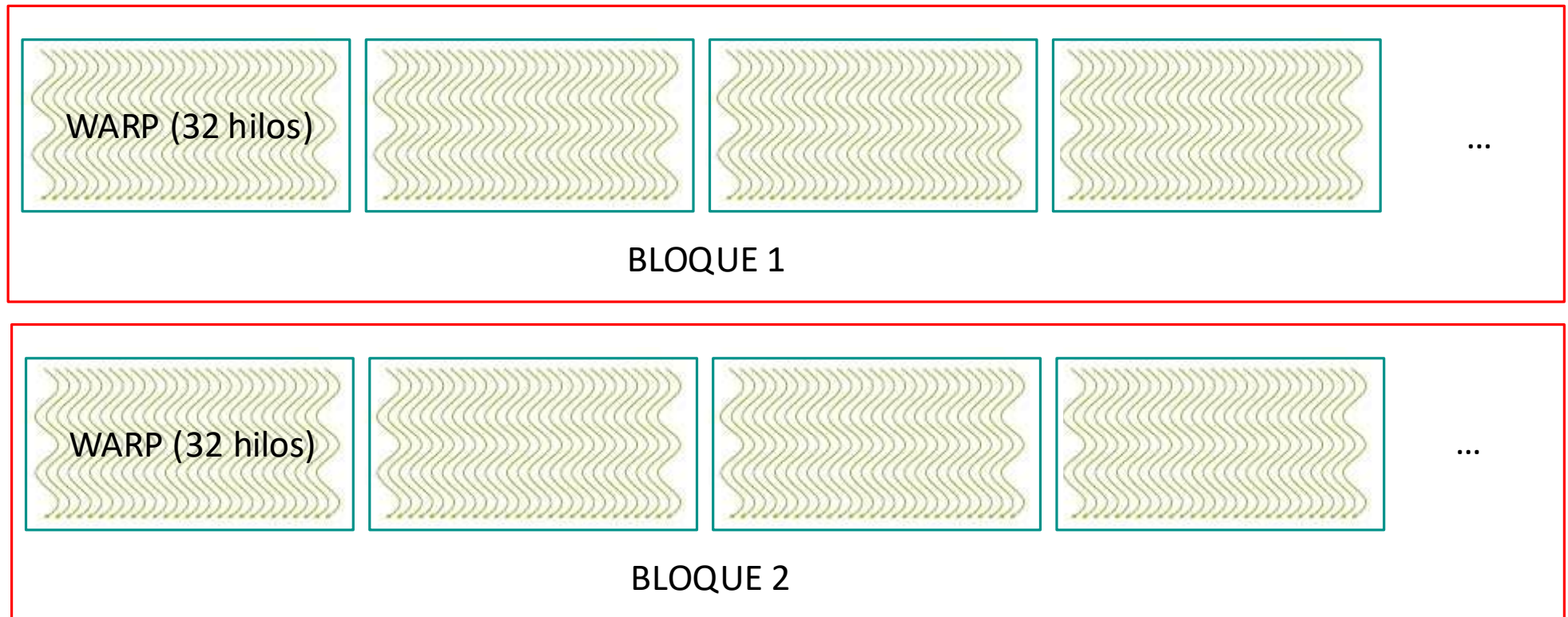
Código serie



Arquitectura NVIDIA

- Hilos agrupados en WARPS y BLOQUES
- 32 hilos -> 1 WARP (Procesamiento SIMD)
- BLOQUES de tamaño variable – Definido por el programador
 - Hilos por bloque múltiplo de 32 para aprovechar recursos
 - Máximo 512 o 1024 hilos (según la arquitectura)

Arquitectura NVIDIA

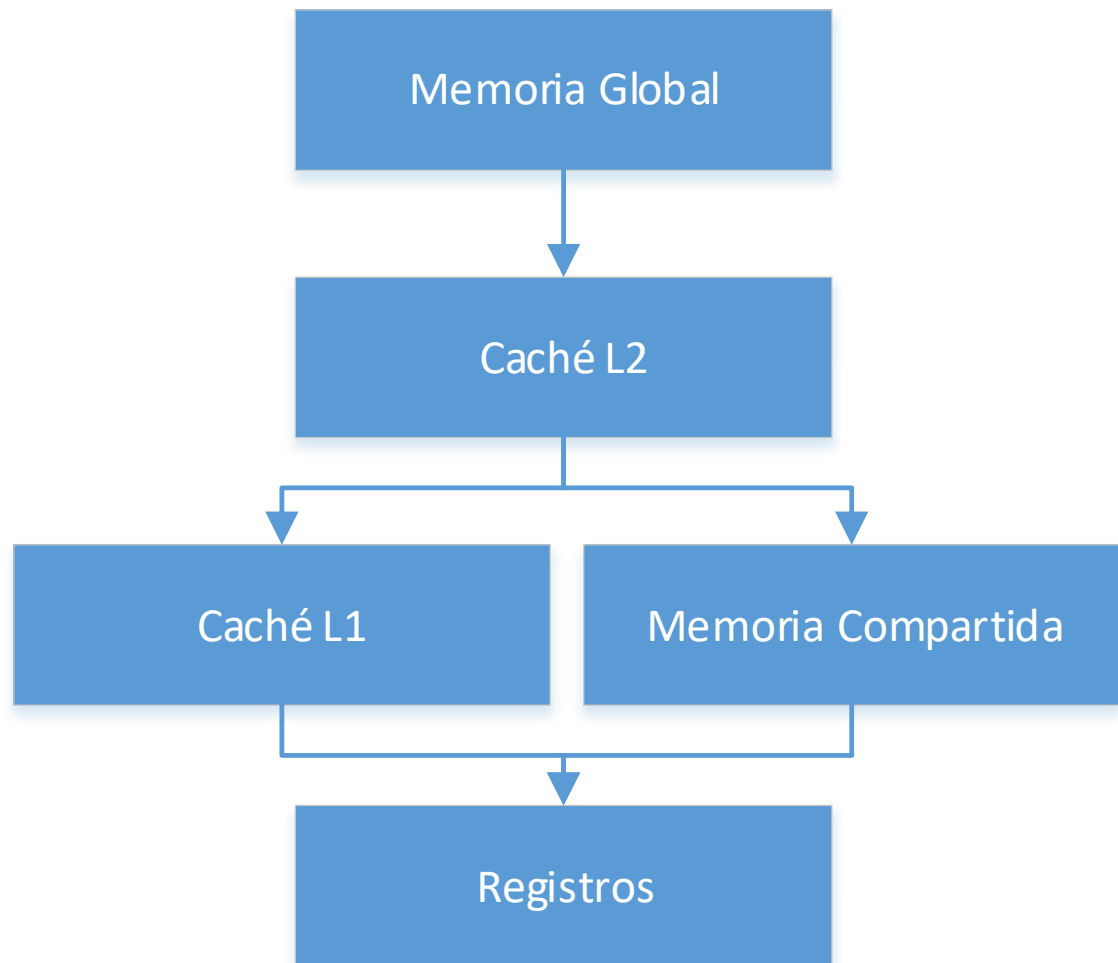


Arquitectura NVIDIA

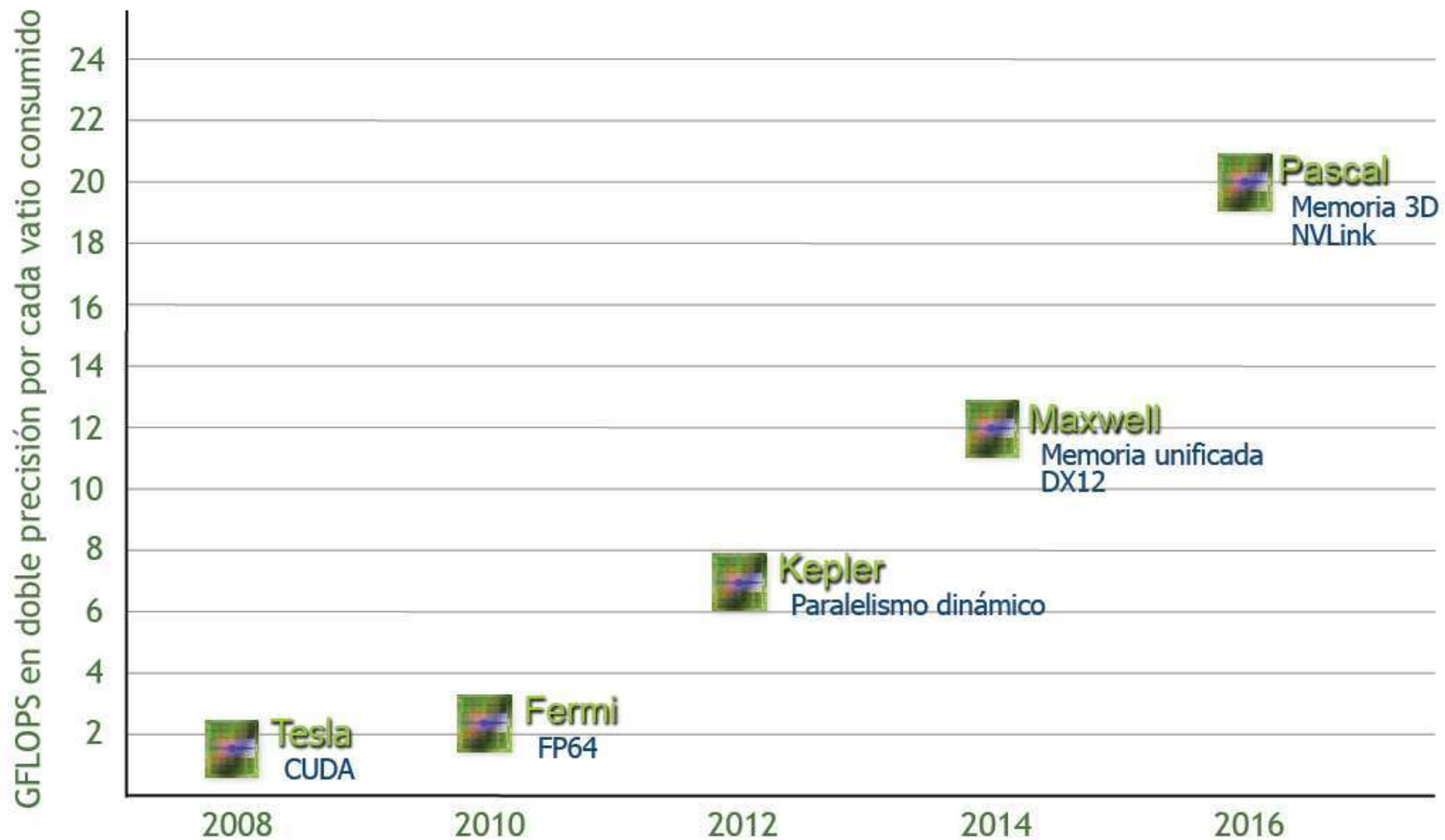
- Las GPUs están formadas por multiprocesadores
 - 1 multiprocesador ejecuta 1 bloque
- Un multiprocesador está compuesto por grupos de 32 núcleos
- Cada grupo de 32 núcleos ejecuta un Warp
 - Ejecuta las mismas instrucciones al mismo tiempo (SIMD)
 - Se debe evitar la divergencia de hilos

Arquitectura NVIDIA

○ Jerarquía de Memoria



Arquitectura NVIDIA

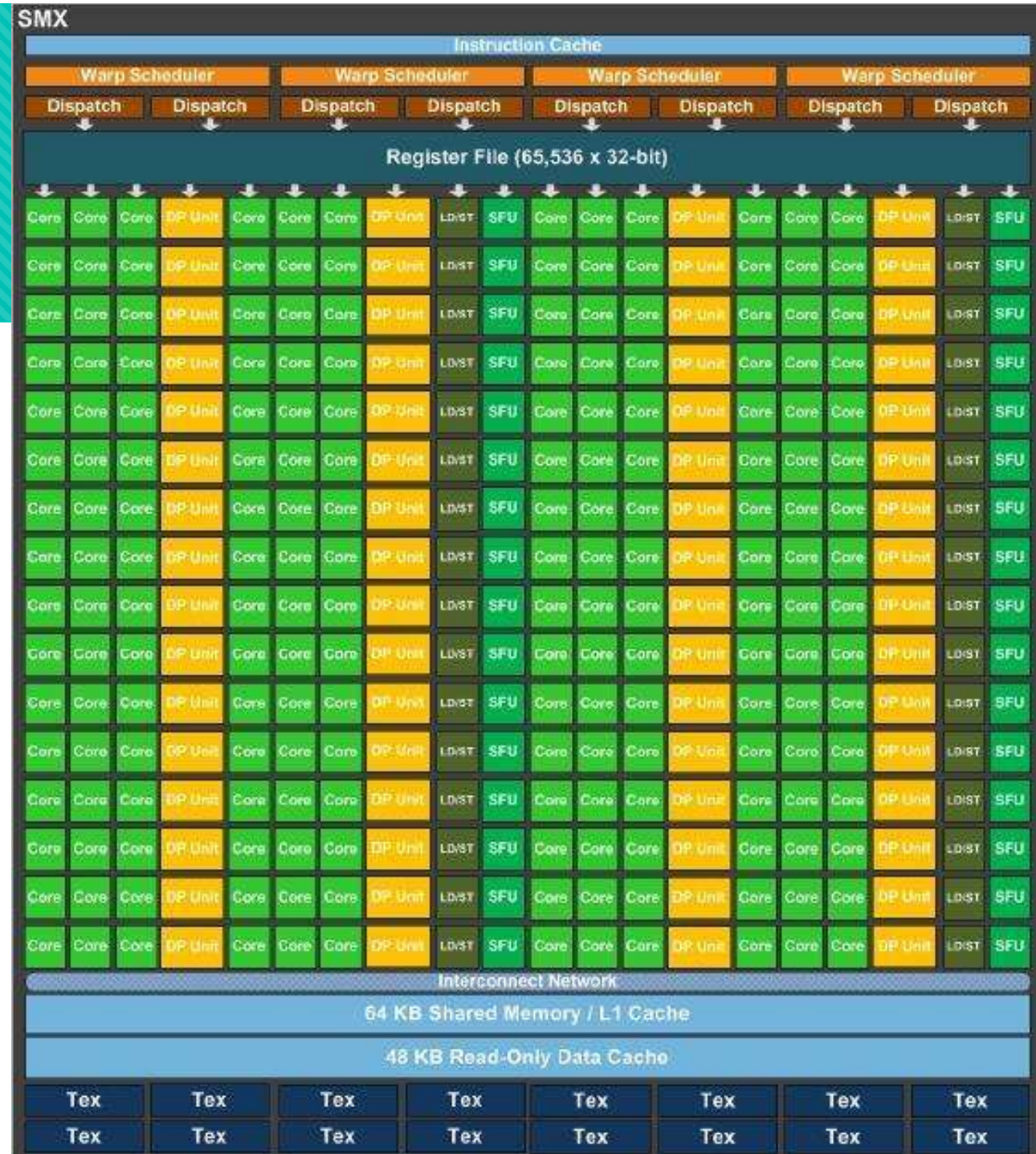


Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - **2.1 Kepler**
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

KEPLER

- Multiprocesador KEPLER
- 192 Núcleos (6 grupos)
- 64 DP Units
- L1 + Memoria Compartida:
 - 48 + 16 KB
 - 32 + 32 KB
 - 16 + 48 KB
- 65 536 registros
 - 341 por núcleo aproximadamente



KEPLER

Chip: GK110



KEPLER

Chip: GK110 – GTX780



Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - **2.2 Maxwell**
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

MAXWELL

○ Multiprocesador Maxwell

○ 128 Núcleos (4 grupos)

○ 4 DP Units

○ Dos cachés L1 de 24KB

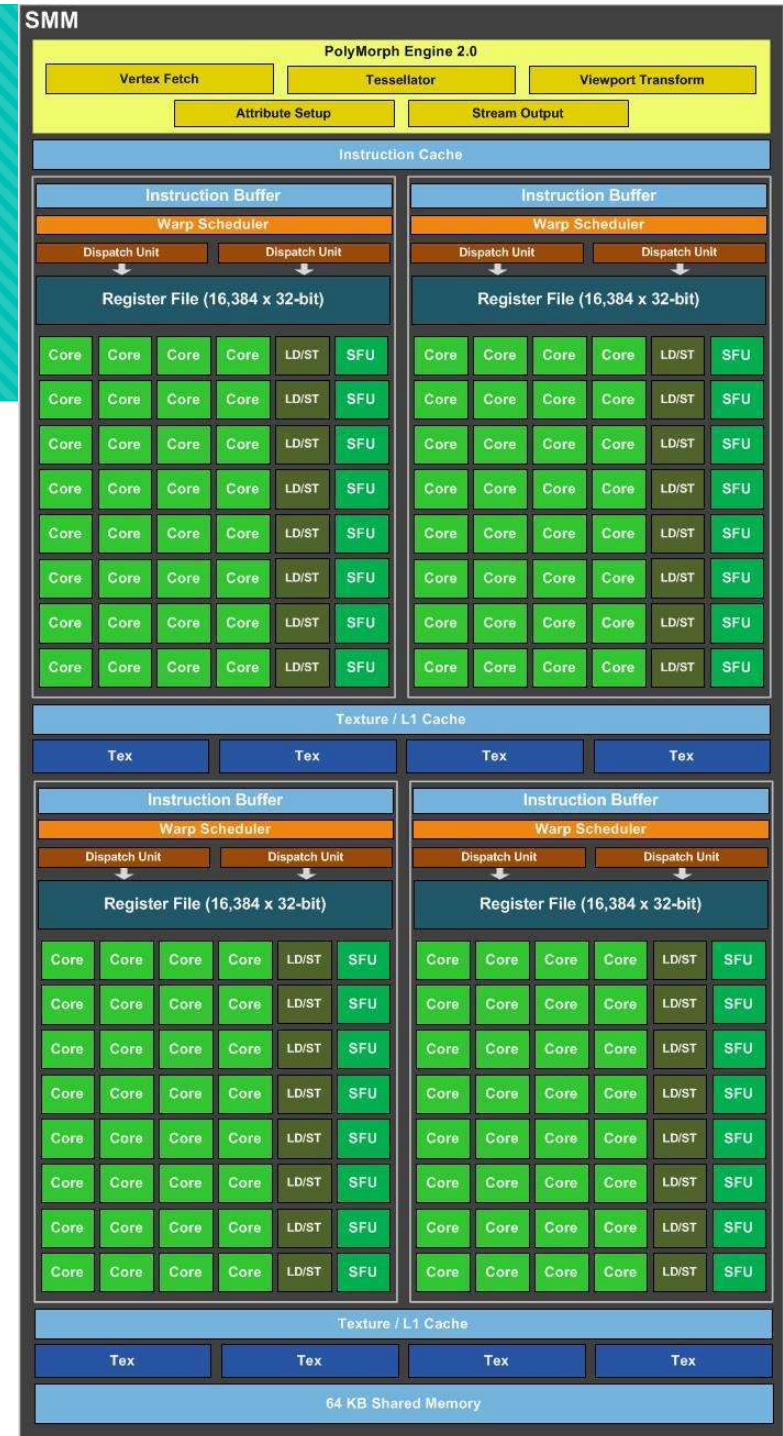
○ Memoria Compartida

○ 64KB – 1ª Gen

○ 96KB – 2ª Gen

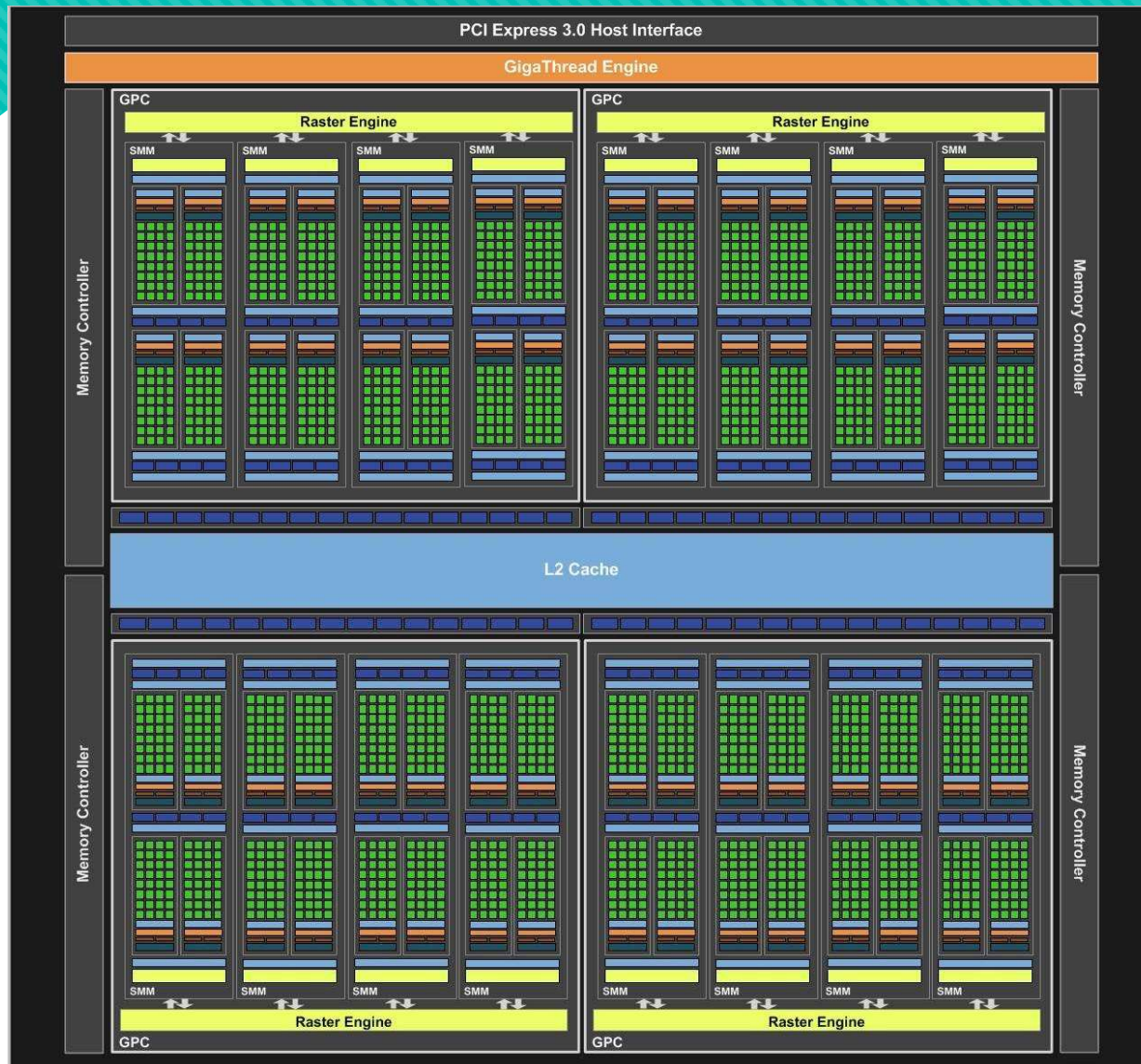
○ 65 536 registros

○ 512 por núcleo



MAXWELL

Chip: GM204 – GTX980



$128 * 16 = 2048$ cores

Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - **2.3 Pascal**
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

PASCAL

- Multiprocesador Pascal
- 64 Núcleos (2 grupos)
- 32 DP Units
- Caché L1 de 24KB
- Memoria Compartida - 64KB
- 65 536 registros
 - 1024 por núcleo



PASCAL

Chip: GP102 – GTX1080 Ti



Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- **3. Programación en CUDA**
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Programación en CUDA

CUDA

CUDA son las siglas de *Compute Unified Device Architecture* (Arquitectura Unificada de Dispositivos de Cómputo) que hace referencia a una plataforma de **computación en paralelo** incluyendo un **compilador** y un conjunto de herramientas de desarrollo creadas por **nVidia** que permiten a los programadores usar una variación del **lenguaje de programación C** para codificar algoritmos en **GPU** de nVidia.

Programación en CUDA

- GPU con arquitectura CUDA
 - Lenguaje de programación
 - Soporte oficial C/C++
 - Wrapper Python – PyCuda
 - Wrapper Java – Jcuda
 - Compilador NVCC + GCC/CL.exe
 - IDE (Editor + compilador + depurador + profiling)
 - Windows: Visual Studio Code + Complemento Nsight
 - UNIX: Nsight Eclipse Edition
- NVIDIA CUDA Toolkit**

Programación en CUDA

Instalación

○ Windows

- Drivers Nvidia (mínimo R390)
- Visual Studio (Recomendado v2015)
- Compilador C/C++ VS
- CUDA Toolkit 9.1

○ <https://docs.nvidia.com/cuda/cuda-installation-guide-microsoft-windows/index.html>

○ UNIX

- Drivers Nvidia (mínimo R390)
- Compilador gcc y g++
- CUDA Toolkit 9.1

○ <https://docs.nvidia.com/cuda/cuda-installation-guide-linux/index.html>

Programación en CUDA

- **HOST** – CPU
- **DEVICE** – GPU
- **COMPUTE CAPABILITY** - Versión que determina las características CUDA disponibles
- **KERNEL** – Programa listo para ser ejecutado en la GPU
- **WARP** – 32 hilos SIMD
- **BLOQUE** – Conjunto de hilos que comparten memoria
- **GRID** – Conjunto de bloques que forman el Kernel
- **CORE** – Ejecuta hilos
- **MULTIPROCESADOR** – Conjunto de cores – Ejecuta bloques

Programación en CUDA

DeviceList.cu

```
1 #include <cuda_runtime.h>
2
3 #include <stdio.h>
4
5 int main(){
6     int deviceCount; cudaGetDeviceCount(&deviceCount);
7     int device;
8     for (device = 0; device < deviceCount; ++device) {
9         cudaDeviceProp deviceProp;
10        cudaGetDeviceProperties(&deviceProp, device);
11        printf("Device %d (%s) has compute capability %d.%d.\n", device, deviceProp.name, deviceProp.major, deviceProp.minor);
12    }
13 }
```

Programación en CUDA

HelloWorld.cu

```
#include <cuda_runtime.h>
```

```
#include <stdio.h>
```

```
__global__ void cudahello(){  
    int thread = threadIdx.x;  
    int block = blockIdx.x;  
    printf("Hola Mundo! Soy el hilo %d del bloque %d\n", thread, block);  
}
```

Device (Kernel)

Número de bloques

Hilos por bloque

```
int main(){  
    cudahello<<<4,4>>>();  
    cudaDeviceSynchronize();  
}
```

Host

Programación en CUDA

- Reserva de memoria:

```
cudaError_t cudaMalloc ( void ** devPtr,  
                          size_t  size  
                          )
```

```
int * device_array;  
cudaMalloc((void **) &device_array, N * sizeof(int));
```

Programación en CUDA

- Copia entre DEVICE y HOST:

```
cudaError_t cudaMemcpy ( void *          dst,  
                        const void *    src,  
                        size_t          count,  
                        enum cudaMemcpyKind kind  
                        )
```

```
cudaMemcpy(device_array, host_array, N * sizeof(int), cudaMemcpyHostToDevice)
```

```
kernelFunction<<<nblocks, nthreads>>>(device_array);
```

```
cudaMemcpy(host_array, device_array, N * sizeof(int), cudaMemcpyDeviceToHost)
```

Programación en CUDA

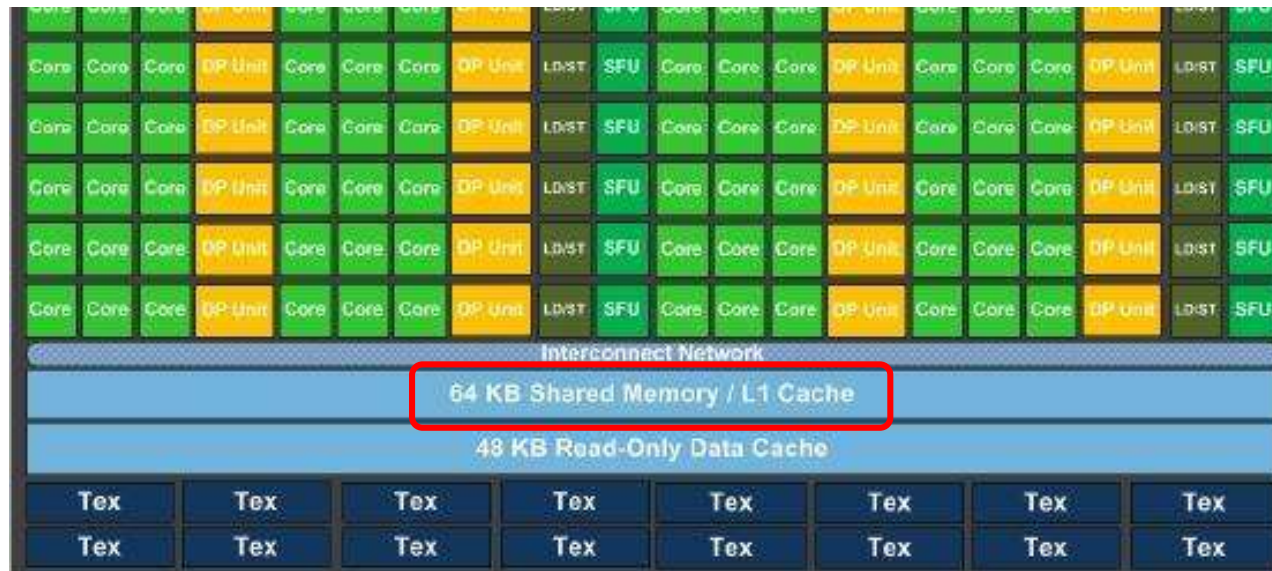
- Liberar memoria:

```
cudaError_t cudaFree ( void * devPtr )
```

```
cudaFree ( device_array );
```

Programación en CUDA

○ Memoria compartida



```
__shared__ int sharedArray[N];
```

Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- **4. Optimización en CUDA**
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - **4.1. Conflictos de bancos**
 - 4.2. Acceso coalescente de la memoria
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Optimización en CUDA

- Acceso **coalescente** a memoria global
 - La memoria global está dividida en segmentos.
 - El acceso simultáneo a posiciones de memoria de un mismo segmento se realiza paralelamente.
 - El acceso simultáneo a posiciones de memoria de distintos segmentos se realiza secuencialmente.
 - `cudaMalloc` alinea el principio de un array con el principio de un segmento.

Optimización en CUDA

CASO 1

- **Hilo 0:** 0, 1, 2, 3
- **Hilo 1:** 4, 5, 6, 7
- **Hilo 2:** 8, 9, 10, 11
- **Hilo 3:** 12, 13, 14, 15

CASO 2

- **Hilo 0:** 0, 4, 8, 12
- **Hilo 1:** 1, 5, 9, 13
- **Hilo 2:** 2, 6, 10, 14
- **Hilo 3:** 3, 7, 11, 15

Optimización en CUDA

CASO 1

- **Hilo 0:** 0, 1, 2, 3
- **Hilo 1:** 4, 5, 6, 7
- **Hilo 2:** 8, 9, 10, 11
- **Hilo 3:** 12, 13, 14, 15

- **Acceso 1:** 0, 4, 8, 12
- **Acceso 2:** 1, 5, 9, 13
- **Acceso 3:** 2, 6, 10, 14
- **Acceso 4:** 3, 7, 11, 15

CASO 2

- **Hilo 0:** 0, 4, 8, 12
- **Hilo 1:** 1, 5, 9, 13
- **Hilo 2:** 2, 6, 10, 14
- **Hilo 3:** 3, 7, 11, 15

- **Acceso 1:** 0, 1, 2, 3
- **Acceso 2:** 4, 5, 6, 7
- **Acceso 3:** 8, 9, 10, 11
- **Acceso 4:** 12, 13, 14, 15

Índice

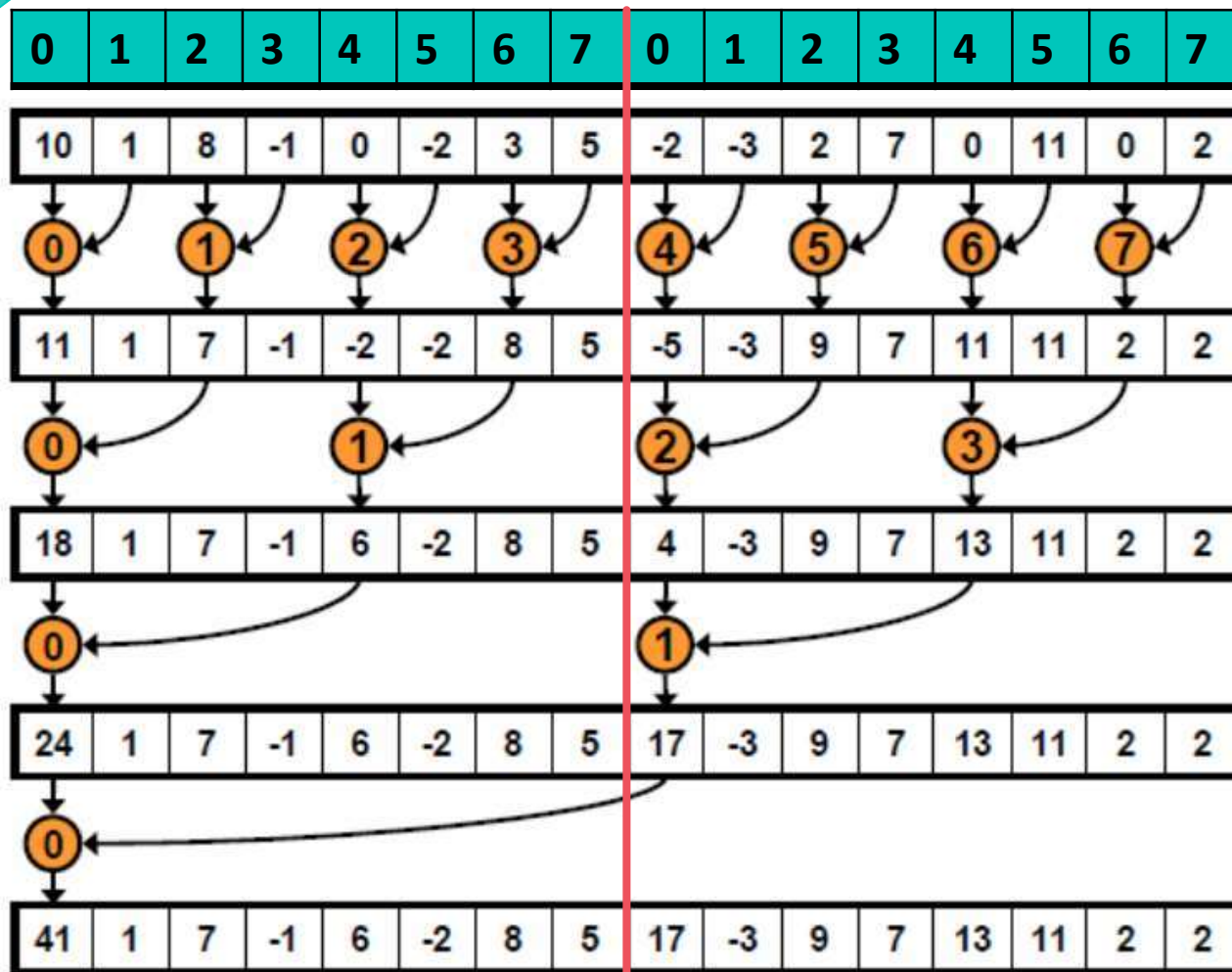
- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - **4.2. Acceso coalescente de la memoria**
 - 4.3. Divergencia de Warp
- 5. Ordenación de Vectores

Optimización en CUDA

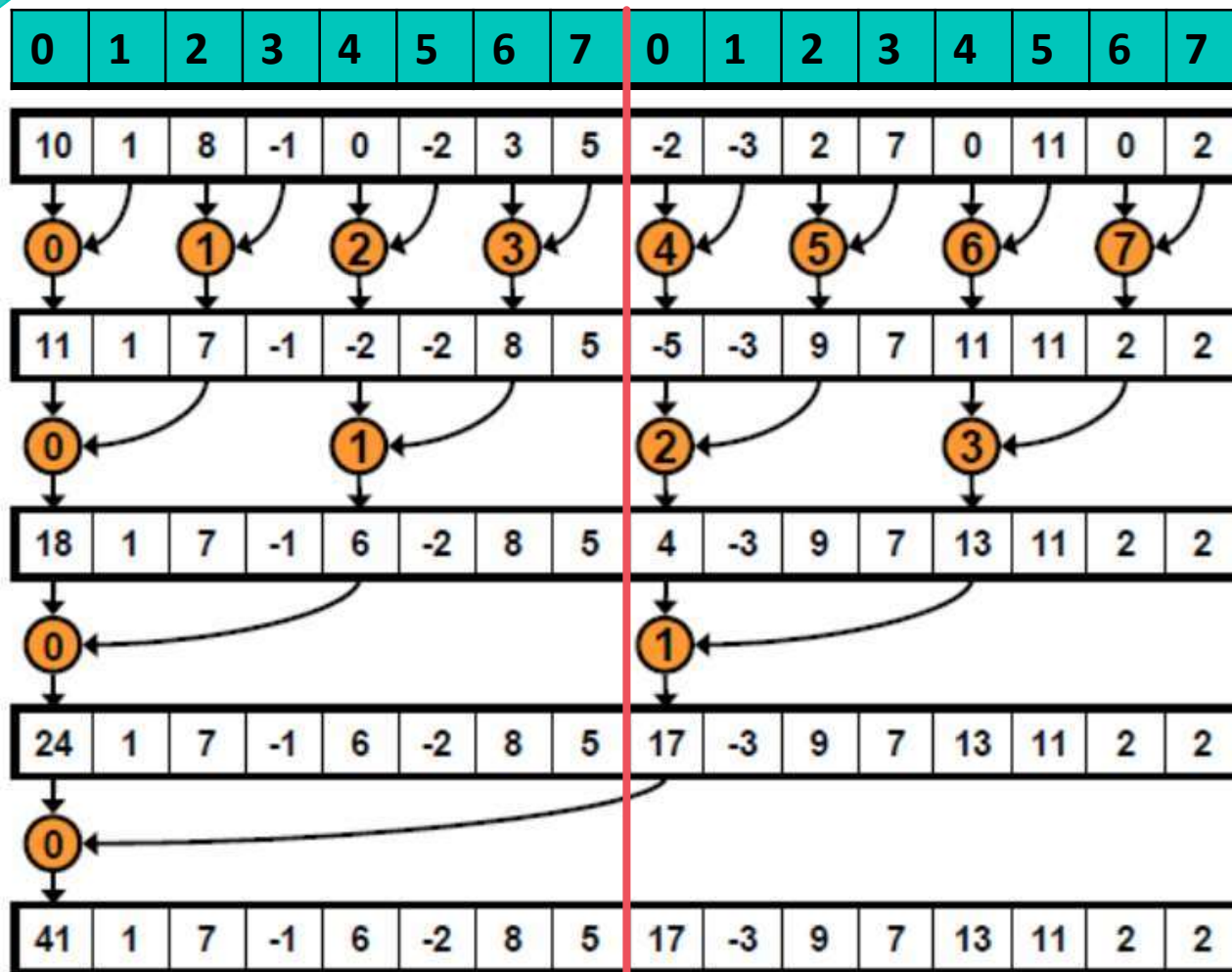
- Conflictos de bancos en memoria compartida:
 - La memoria compartida está dividida en 32 bancos de memoria.
 - Cada banco puede leer/escribir una palabra de 4 bytes en un solo ciclo.
 - Varios accesos a distintos bancos de memoria se realizan paralelamente.
 - Varios accesos al mismo banco de memoria se realizan secuencialmente.

[illegible]

Optimización en CUDA



Optimización en CUDA



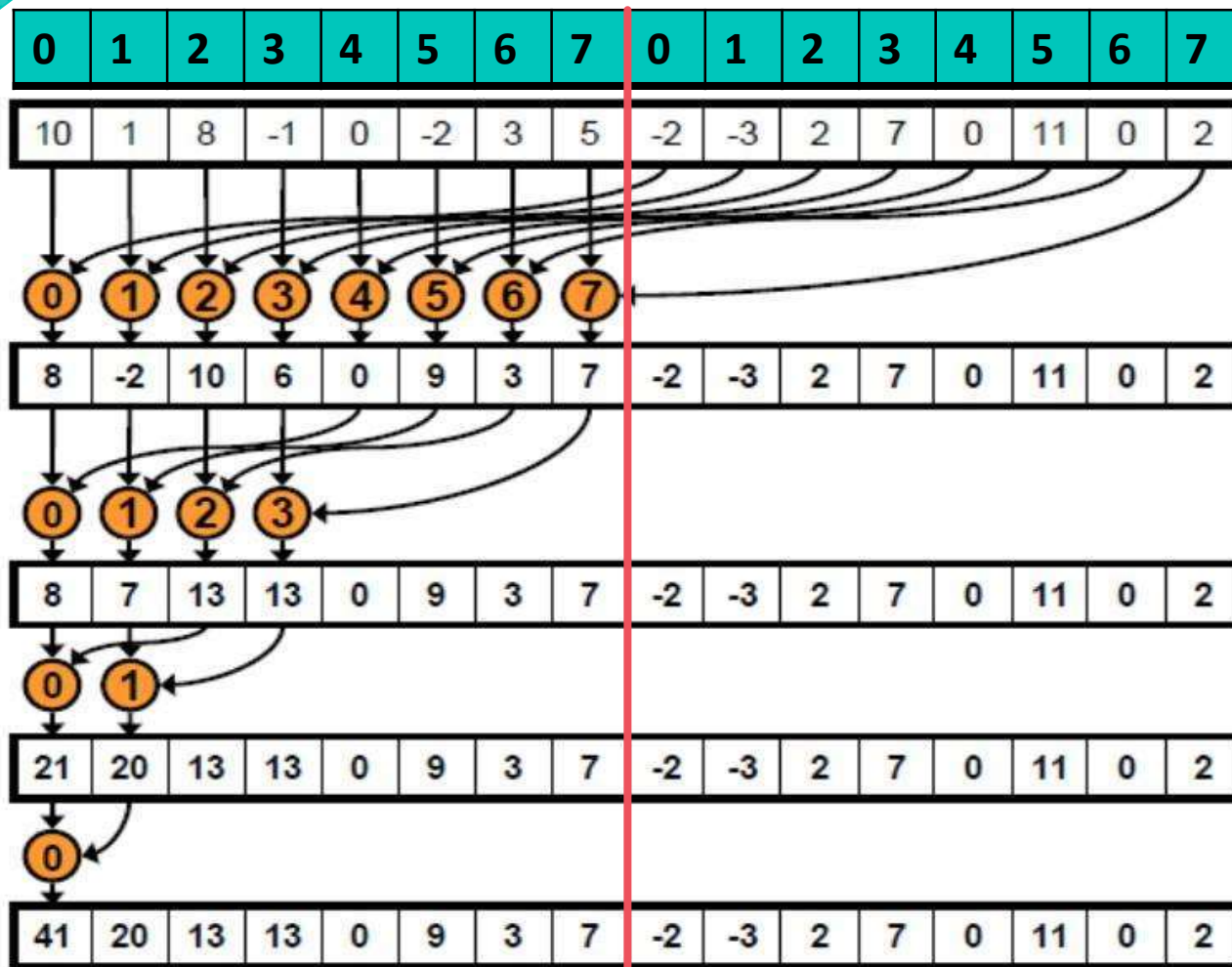
Acceso 1: 0, 2, 4, 6, 0, 2, 4,
Acceso 2: 1, 3, 5, 7, 1, 3, 5,

Acceso 1: 0, 4, 0, 4
Acceso 2: 2, 6, 2, 6

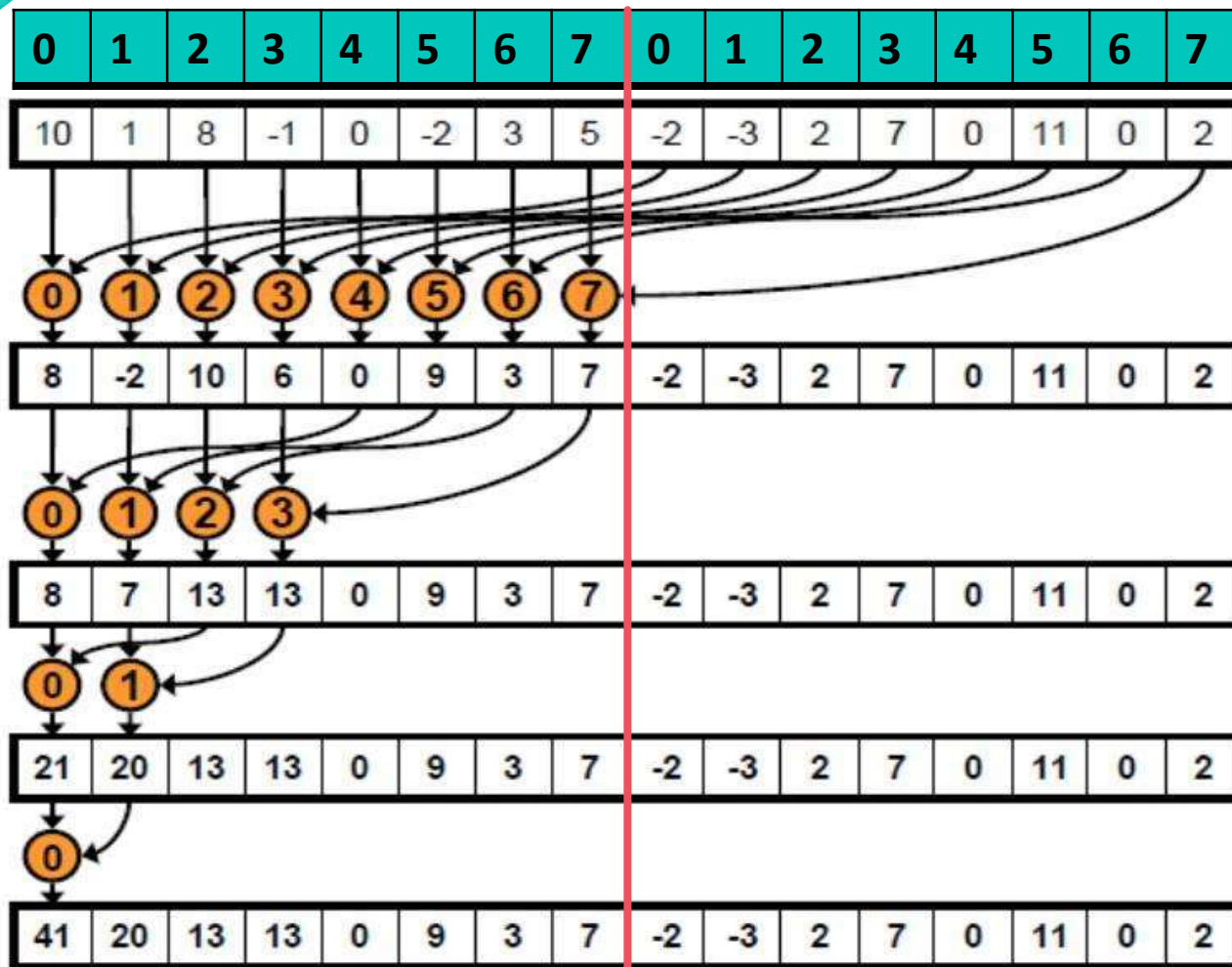
Acceso 1: 0, 0
Acceso 2: 4, 4

Acceso 1: 0
Acceso 2: 0

Optimización en CUDA



Optimización en CUDA



Acceso 1: 0, 1, 2, 3, 4, 5, 6,
Acceso 2: 0, 1, 2, 3, 4, 5, 6,

Acceso 1: 0, 1, 2, 3
Acceso 2: 4, 5, 6, 7

Acceso 1: 0, 1
Acceso 2: 2, 3

Acceso 1: 0
Acceso 2: 1

Índice

- 1. Computación Heterogénea
- 2. Arquitectura Nvidia
 - 2.1 Kepler
 - 2.2 Maxwell
 - 2.3 Pascal
- 3. Programación en CUDA
- 4. Optimización en CUDA
 - 4.1. Conflictos de bancos
 - 4.2. Acceso coalescente de la memoria
 - **4.3. Divergencia de Warp**
- 5. Ordenación de Vectores

Optimización en CUDA

- Divergencia de Warp
 - Ejecución de un Warp -> SIMD
 - Si el flujo de programa se divide en dos caminos, estos se ejecutan secuencialmente.
 - Se debe evitar if y switch.

Optimización en CUDA

○ Divergencia de Warp

- Ejecución de un Warp -> SIMD
- Si el flujo de programa se divide en dos caminos, estos se ejecutan secuencialmente.
- Se debe evitar if y switch.

```
if (var < 10) {  
    var = var*4;  
}else{  
    var = var/5;  
}
```

Optimización en CUDA

○ Divergencia de Warp

- Ejecución de un Warp -> SIMD
- Si el flujo de programa se divide en dos caminos, estos se ejecutan secuencialmente.
- Se debe evitar if y switch.

```
if (var < 10) {  
    var = var*4;  
}else{  
    var = var/5;  
}
```

```
var = (var<10)*4*var + (var>=10)*var/5
```